

```

import argparse import os from datetime import datetime import math import shutil

import Generator import BaseTypeGenerator from packets import packets

def read_version(): return int(open("latest_rem_version.txt", "r").read()) def write_version(version):
open("latest_rem_version.txt", "w").write(str(version % 16))

parser = argparse.ArgumentParser() parser.add_argument('-version', help='REM version number',
type=int)

args = parser.parse_args()

version = read_version() + 1 if args.version is not None: version = args.version

print(f"Generating REM version {version}")

generator_c = Generator.C_Generator() generator_python = Generator.Python_Generator()
generator_proto = Generator.Proto_Generator()

os.makedirs("generated_c", exist_ok=True) os.makedirs("generated_python", exist_ok=True)
os.makedirs("generated_proto", exist_ok=True)

for packet_name in packets: variables = packets[packet_name]

filename = f"{packet_name}.h" with open(os.path.join("generated_c", filename), "w") as file:
file.write(generator_c.generate(packet_name, variables)) print(f"Generated file {filename}")

filename = f"{packet_name}.py" with open(os.path.join("generated_python", filename), "w") as file:
file.write(generator_python.generate(packet_name, variables)) print(f"Generated file {filename}")

filename = f"{packet_name}.proto" with open(os.path.join("generated_proto", filename), "w") as file:
file.write(generator_proto.generate(packet_name, variables)) print(f"Generated file {filename}")

basetypegenerator_c = BaseTypeGenerator.C_BaseTypeGenerator() basetypegenerator_python =
BaseTypeGenerator.Python_BaseTypeGenerator()

filename = "REM_BaseTypes.h" with open(os.path.join("generated_c", filename), "w") as file:
file.write(basetypegenerator_c.generate(packets, version)) print(f"Generated file {filename}")

filename = "REM_BaseTypes.py" with open(os.path.join("generated_python", filename), "w") as file:
file.write(basetypegenerator_python.generate(packets, version)) print(f"Generated file {filename}")

shutil.rmtree("../include") os.makedirs("../include", exist_ok=True) for file in os.listdir("generated_c"):
shutil.move(f"generated_c/{file}", f"../include/{file}")

shutil.rmtree("../python") os.makedirs("../python", exist_ok=True) for file in
os.listdir("generated_python"): shutil.move(f"generated_python/{file}", f"../python/{file}")

shutil.rmtree("../proto") os.makedirs("../proto", exist_ok=True) for file in
os.listdir("generated_proto"): shutil.move(f"generated_proto/{file}", f"../proto/{file}")

write_version(version)

print("\n") print(f"All files generated with REM version {version}") print("If you want to generate
using a specific version, run 'python main.py -version %d'")

```